

반복되는 운영을 줄이고 싶었습니다.

처음엔 kubectl, 콘솔, 알림을 왔다 갔다 하는 게 번거로웠습니다. 그래서 자주 하는 일을 묶고, 상태가 남고, 되돌릴 수 있는 도구를 만들었습니다.

Cho YunHo

운영 자동화 · 플랫폼 엔지니어링

FastAPI

Kubernetes

Docker

Proxmox

출발점은 단순했습니다.

같은 작업을 두 번 하고 싶지 않았고, 실행한 일은 나중에 확인할 수 있어야 한다고 생각했습니다.

01

K-Le-PaaS

배포, 롤백, 알림, 헬스체크를 한 곳에서 처리하려고 만들었습니다.

02

Heimdall

브랜치가 바뀌면 미리보기 환경이 뜨고, 로그와 롤백이 남게 했습니다.

03

Gjallar

VM 운영은 실수 비용이 커서, 추천과 실행을 분리하고 승인 단계를 뒀습니다.

kubectl과 콘솔을 오가는 일을 줄이고 싶었습니다.

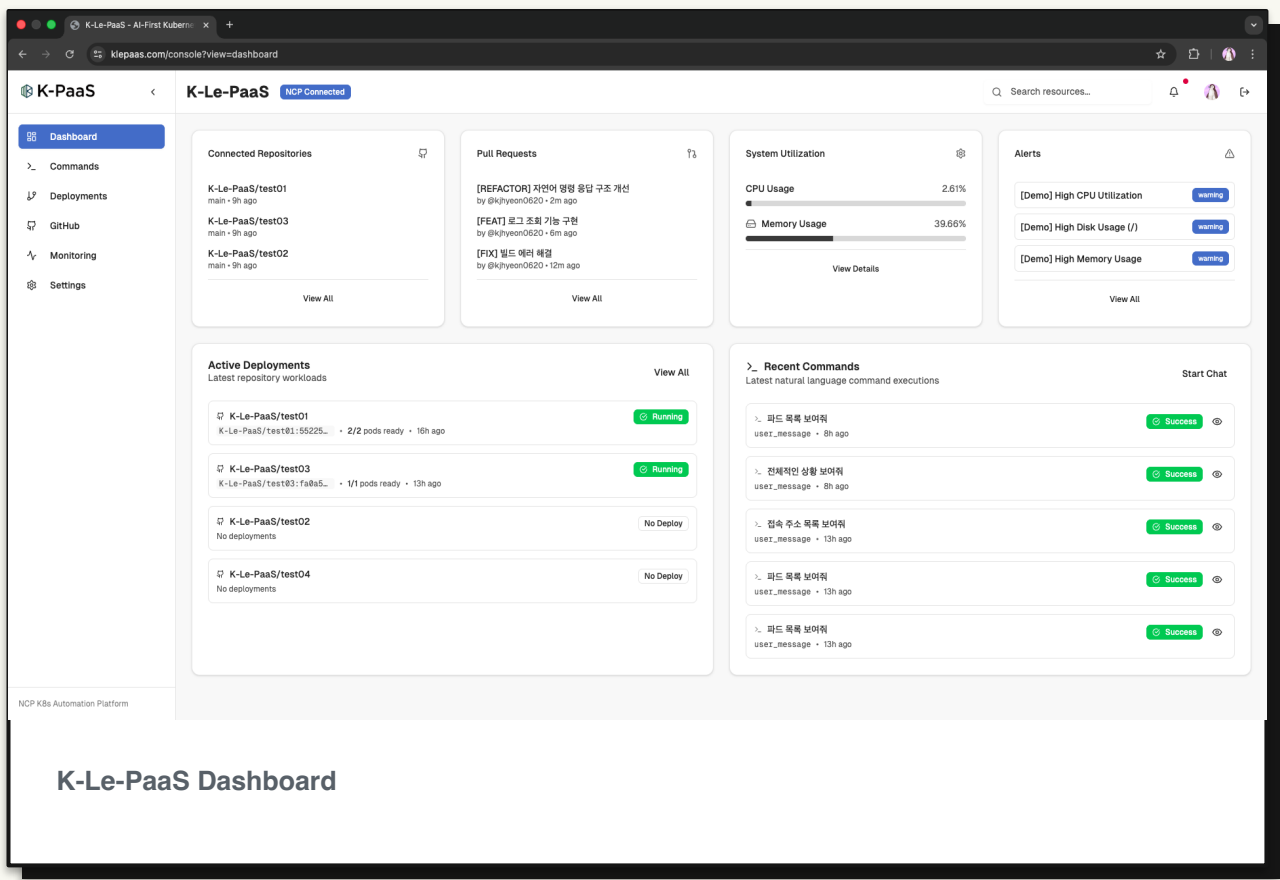
K-Le-PaaS는 GitHub, Kubernetes, NCP, Slack, 모니터링을 FastAPI 백엔드로 묶은 프로젝트입니다. 배포하고, 롤백하고, 상태를 보고, 알림을 받는 일을 한 흐름으로 줄였습니다.

- 불편

명령어, 콘솔, 알림이 따로 있어서 매번 확인할 곳이 많았습니다.
- 구현

자연어 명령, Webhook, Kubernetes/NCP 작업, Slack 알림을 백엔드로 묶었습니다.
- 결과

누가 무엇을 실행했는지 남고, 같은 운영 작업을 더 짧게 처리할 수 있게 했습니다.



K-Le-PaaS Dashboard

푸시한 뒤 “어디서 확인하지?”를 없애고 싶었습니다.

Heimdall은 GitHub/GitLab 프로젝트를 등록해두면 브랜치 변경을 감지하고, Docker 기반 미리보기 환경을 띄워주는 배포 매니저입니다.
상태, 로그, 히스토리, 롤백까지 한 화면에서 확인하게 했습니다.

1

Register

프로젝트와 브랜치를 한 번 등록

2

Build

Webhook을 받으면 Docker 이미지 빌드

3

Preview

URL, 상태, 로그를 바로 확인

4

Rollback

문제 있으면 이전 이미지로 되돌림

제품 관점

코드를 올린 뒤 결과를 따로 찾아다니지 않고, 지금 어떤 버전이 떠 있는지와 되돌릴 수 있는지를 바로 보게 만들었습니다.

위험한 작업은 한 번 더 멈추게 하고 싶었습니다.

Gjallar는 Proxmox 운영을 다루려고 만든 콘솔입니다. VM 이동 같은 작업은 자동으로 바로 실행하기보다, 현재 상태를 보고 추천을 만들고, 승인한 뒤에만 실행되게 나눴습니다.

Check

상태 확인

VM, node, storage, HA, task 상태를
Proxmox에서 먼저 읽습니다.

Suggest

추천 만들기

CPU/Memory, 정책, 리스크를 보고 이동해도 되
는지 판단합니다.

Approve

승인 후 실행

실행은 승인과 live check를 통과한 job에서만
진행합니다.

제품 관점

자동으로 다 해주는 것보다, 실수하면 큰 작업은 사람이 확인하고 승인할 수 있게 만드는 쪽에 집중했습니다.

제가 만든 것들의 공통점은 반복 작업을 줄이는 것입니다.

자주 하는 일 묶기

배포, 롤백, 알림, 상태 확인처럼 반복되는 일을 한 화면과 API로 묶습니다.

결과 남기기

실행한 일의 로그, 히스토리, 현재 상태를 남겨 나중에 다시 볼 수 있게 합니다.

위험한 일 멈추기

VM 이동처럼 실수 비용이 큰 일은 바로 실행하지 않고 확인과 승인을 거치게 합니다.

도구 사이 간격 줄이기

kubect!, Proxmox, NCP, Slack처럼 떨어진 도구를 사용 흐름에 맞게 연결합니다.

CLOSING

자동화는 결국
사람이 덜 헛갈리게
만드는 일이라고
생각합니다.

K-Le-PaaS, Heimdall, Gjallar는 모두 같은 불편함에서 시작했습니다. 매번 반복하는 일을 줄이고, 결과를 남기고, 위험한 일은 한 번 더 확인하게 만드는 것. 그런 도구를 계속 만들고 있습니다.

GITHUB

github.com/CodingPenguin-yoon

PROJECTS

K-Le-PaaS · Heimdall · Gjallar